PROJECT-BASED LEARNING

PHASE-I EVALUATION

SmartFile : C++ Based File Compression and Decompression Using Huffman Coding

Team ID: DSCPP-III-2026-T056

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T056
Semester: 3rd
Section: DS1,ML1,ML-2,SEC-H
Domain: *XXXX*

Team Members

1. *Rishi Negi – 2510380076*
2. *Vardan Bansal – 2510372517*
3. *Priyansh Rautela – 2510370501*
4. *Pragya Saksham – 2510370412*

Mentor

Dr.Hemant Singh Pokhariya

Interactions completed: 3

Problem Statement & Motivation

Problem Statement

To develop a C++ based file compression and decompression system that reduces file size using HuffMan coding while allowing the original file to be recovered without data loss

Motivation

- *Growing number of digital files*
- *Storage and transfer requirements increase*
- *Compression saves storage and data*
- *Demand for lossless compression*

Target Users / Stakeholders

- *Students and Academics*
- *General Users*
- *Organisations and Developers*

Objectives

Project Objectives

1. *Develop a file compression and decompression solution.*
2. *Implement Huffman Coding for efficient compression.*
3. *Apply DSA and OOP concepts in C++.*
4. *Test compression and decompression using sample files.*
5. *Evaluate storage savings and system performance*

Key Objective: *Achieve effective file compression and decompression while preserving the original data.*

Proposed Solution

Proposed Approach

1. *File Selection*
2. *Byte Frequency Analysis*
3. *Huffman Tree Construction*
4. *Bit-Level Encoding*
5. *Compressed File Generation*
6. *Decode and Restore File*

Key Features

- *File Compression*
- *File Decompression*
- *Huffman Coding*
- *Binary File Handling*
- *Compression Statistics*
- *Lossless Data Recovery*

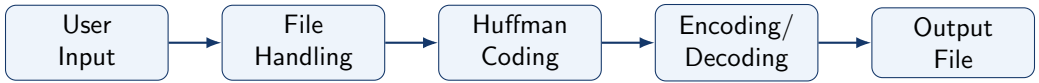
SmartFile addresses the problem of increasing file storage and transfer requirements by reducing the size of files using Huffman Coding. It saves storage space and reduces transfer requirements while preserving the original data through lossless decompression.

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures	*Arrays, Linked Lists, Trees, Priority Queue*	*HuffMan Module*
OOPs with C++	*Classes,Object,Encapsulation,Function*	*Compression and Decompression*

Requirement: Demonstrate the practical integration of data structures and OOPS in SmartFile compression and decompression system.

System Architecture / Workflow



Major Components

- *File Handling Module*
- *Huffman Coding Module*
- *Compression / Decompression Module*
- *Bit Processing Module*
- *User Interface Module*

Data / Control Flow

- *File -▶ Frequency -▶ Huffman Tree*
- *Compression and decompression modules interact*
- *Compressed / restored file output*

Technology Stack & Methodology

Technology Stack

- Programming: *C++*
- Database: *Not Required*
- Tools / Framework: *Huffman Coding,File Handlling*
- IDE: *Visual Studio Code*
- Version Control: *Git / GitHub*

Development Methodology

1. Requirement Analysis
2. System Design
3. Huffman Implementation
4. Testing and Debugging
5. Performance Evaluation

C++ is selected for efficient file handling and implementation of DSA algorithm.Huffman Coding provides efficient compression.VS code provides lightweight development environment,while Git/Github helps in version control and collaboration

Initial Progress & Team Contribution

Progress Achieved

- *Background / literature study*
- *Requirements identified*
- *Architecture prepared*
- *System design prepared*
- *Initial coding started*

Individual Contribution

Member	Role	Contribution
Rishi Negi	*Lead*	*Project planning and coding*
Vardan Bansal	*Developer*	*Huffman Tree and Compression*
Priyansh Rautela	*Testing*	*Testing and error checking*
Pragya Saksham	*Developer/Docs*	*Coding and documentation*

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal and Design
2. Phase-II: Development
3. Phase-III: Testing and Final Implementation

Expected Outcomes

- *Working compression and decompression system.*
- *File size and space-saving results.*
- *Useful for different types of files.*
- *Easy to improve in the future.*

References

1. *Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes.*
2. *Weiss, Mark Allen. Data Structures and Algorithms in C++.*
3. *C++ Reference Guides , including cppreference.com , for std :: priority_queue and file handling functions.*

Thank You
Questions & Suggestions